

小芽：宠物对战游戏里的 LLM 教练

设计与实现说明

腾讯 IEG 面试题提交稿 2026 年 9 月



本报告由 AI 生成，由本人验收。正文引用的每个数字都对应仓库里一份可复现的实验产物，结论、失败与未完成项经本人逐条核对后写入；没有实测过的部分，文中直接写明没测过。

1 我怎么理解这道题，以及先砍掉了什么

题目给的场景是宠物对战游戏：玩家养一批精灵，打 PVP，配阵容，日常培养。要交付的是一个 LLM 教练，具备军师、陪练、老师三种角色感，至少完整闭环其中一项，其余可以只做设计。参考方向列了 Agentic RL、RAG、ReAct、Memory Stream / Reflection，产品参考是和平精英的“小田”和王者荣耀的“灵宝”。

拿到题目我先做的不是写代码，而是划边界。三件事我判断为本次不做，理由写在下面——不做的决定和做的决定一样需要交代。

不做三渲二的美术和完整游戏内容。题目要的是教练，游戏是产生局面的环境。我把宠物控制在 12 只、5 个关卡，够产生“抢先收尾还是留能量”“换宠承伤还是原地治疗”这类真实取舍就停手。继续堆宠物不会让教练更聪明，只会让平衡工作吃掉全部时间。

不做 LLM 权重训练。本机是 arm64，没有可用的 CUDA 后端，也没有云上 GPU 预算。我做了两个能在这台机器上真跑完的小训练实验，它们各自是什么、不是什么写在“数字”一节里，我没有把它们包装成“训练了模型”。



不做联网 PVP。本地对战是同一台设备上分屏轮流操作。它离真实竞技还有距离，这一点在“还没做完的”一节里明说了，没有含糊过去。

三个角色现在都做到了可运行的闭环。军师和老师是最先做完的；陪练最初只交了设计文档，后来又补齐了实现。补了什么、边界在哪，下一节分开讲。

2 怎么看这个东西

本地跑起来：

```
cd pet-coach-game
npm start          # Node.js 20+
```

浏览器打开 <http://127.0.0.1:8765/>。想看模型能力，先在 `/connect.html` 里录入 DeepSeek 密钥  密钥只留在后端进程内存里，不落盘。不录也能玩——规则建议、复盘、培养建议和小测都不依赖模型。

进页面是营地，顶部两个入口。**训练**按关卡打固定对手，教练全程给建议；**对局**里对手从全部 12 只里自由配队、等级按你的队伍适配，教练照常工作。对局还有第二种对手：真人同机分屏，两个人各选各的，两边各有一条教练。

营地在还没选之前不替你选  顶部那个徽标原来一进页面就写着“训练·PVE”，等于替你认定了模式；现在它的默认值是未选择，显示“营地·v0.11”，要等你按下出征才变成对应的模式名。

想直接看到教练在做什么，有两处最省事：战斗页底部左右两侧各有一条教练条，各自只分析自己那一侧的局面；打完一局后自动出现整局复盘，会指出具体是哪一回合、当时还有哪些别的选择。

3 三个角色各自做到哪一步

3.1 军师：算和说分开，判断留在代码里

军师是这次的重心。它的工作方式是把“算”和“说”切开：伤害、行动顺序、合法性全部由游戏引擎枚举，模型不参与计算，只负责把比较结果讲成玩家能理解的取舍。 

玩家面对一个局面时，引擎枚举双方所有合法行动，对每一种组合跑一遍共享计算器，得到平均收益和最坏分支；双方同速时，两种出手顺序各算一次，这样搜索不会偷偷利用随机数。模型拿到的是一份算好的事实包，里面有数字、每张战术卡的适用条件和反例。它的任务只是解释。

这个分工为什么必要，一个例子就够。同一个残血目标，火花直接伤害 52，追猎 75，两个都够击倒，都耗 2 点能量。只看面板，很自然会告诉玩家“选追猎”。但这是错的：追猎的额外伤害来自“目标已灼烧”这个前提，对方吃药、换宠、防御都会让前提失效。这种判断交给模型是猜，交给枚举是算。

“该不该调工具”这件事，我最后也没让模型做。证据包里本来就带着当前局面和最近回合事件，“我还剩多少血”“对面还有几瓶药”这类问题的答案已经在包里，不值得为它多跑一次往返；真正需要查的只有证据包结构上不含的四类：指定回合的原始事件、两个具体行动的分支模拟、整局分页统计、以及包里没有的战术规则。判定需要时，代码直接执行第一步，之后才问模型要不要继续查；判定不需要时，整个工具循环跳过。这是三轮真实评测之后才改的，过程在“难点”一节。

局内主动提示同样归军师，不归陪练——这一点我改过一次。触发条件写在代码里：宠物倒下；犹豫不决（在两个以上选项之间来回至少 3 次、悬停累计 8 秒）；明显策略错误（真实枚举出来的分差大于 5，且确实造成了后果）；长停留（同一个选项上停满 10 秒且指针还在选项区）；以及这一轮补的危险血线——血量低于 35% 且背包里还有药时开口，说“血量偏低，背包里还有 N 个回复药”。最后一条有两个不该开口的边界：满血不提，**没药也不提**，因为那时没有可选项，提了只是唠叨；提醒的价值在于“你忘了还有一个选择”。安静档和“本局不再提醒”永远优先，不被任何推断覆盖。

属性提示这一轮改的是措辞，理由值得写下来。原先只说“当前处于属性劣势”，现

在说“属性被克：它打你更疼，你打它也减伤，可以考虑换一只不被克的”。改成对称表之后，“我打它 $\times 0.75$ ”与“它打我 $\times 1.5$ ”是同一个条件的两面，所以不能加第二条理由——只能把唯一那条说完整。而只说一半的时候，玩家读到的是“它打我疼”，读不到“我打它也打不动”，可后者才是“换一只”这个动作的直接依据。

3.2 老师：复盘只看当时能看到的

复盘分整局和单回合两层。整局先给行动分布和回合数，再挑出造成减员或血量大幅变化的回合；单回合用回合开始时的局面做条件比较，比如“当时目标 78 血，火花不防御打 24、防御打 8，单次攻击不足以收尾”。

有一条规则是硬的：复盘一律使用回合开始时的公开状态，不使用对手实际打出的那一招。理由很直白——如果拿事后结果当依据，任何一次失败都能被解释成“你当时选错了”，这既不公平，也没法教人。这条有测试保证：改掉对手的实际行动，复盘结论不变。

练习跟着复盘走。玩家在某个回合做了选择，教练就这个选择对应的知识点出一道题（灼烧追击、防御节奏、换宠承伤、先制与速度四类）。出题、等待作答、判题、取消这四个状态由程序控制，模型不能提前报答案，也不能把题改写成别的。题目和玩家刚才的真实选择挂钩，不是随机题库。

3.3 陪练：先只有设计，后来补上了实现

这一块我改动最多，先说现状。

陪练的状态不是模型编出来的，是从真实对战历史派生的：连败几局、最近有没有赢、这台机器上有没有历史记录。从状态派生出语气档位，共四档，每档有字数上限和问句上限。档位不是装饰——它会作为证据写进模型的输入，档位不同，输出的长度和语气跟着变。有两条底线：本机没有任何真实记录时返回最低档并且不编造过去；某个档位需要的事实一条都拼不出来时，明说降到最短承接句，而不是硬凑一句符合档位的漂亮话。

主动开口只挂在两个事件上，每局各一次：本局第一个同伴倒下，以及整局结束。加起来一局最多两次。

教学账本也是这一段补的。已经教过的知识点，没有新的反证就不再重复讲；但军师在局内反复观察到同一个知识点上的失误（3 次独立行动都不合理）时，会把这一课标回未掌握，老师于是有机会再讲一次，而且说得出“为什么又教”。叉掉是分角色的：叉掉教学类，教学安静但军师类照常；两类都被叉掉，才整体静默。这个分层是刻意的——玩家点叉的意思通常是“这个别讲了”，不是“你闭嘴”。

情绪这件事我做得比设计文档保守。原因是我对“给游戏助手加情绪”本身有保留：项目调研过的灵宝资料里有一条我认同的判断，情绪陪伴很容易变成油腻，玩家不打开聊天，不代表需要被安慰。所以我给它加的约束比一般助手更严——允许沉默、不主动提问、不空泛安慰、不因为一次失败就弹话。落到实现里就是那四档语气加字数

上限，不是一套情绪状态机。所以如果问“陪练有没有情绪”，准确答案是：有一层受约束的语气调节，没有情绪状态机。

4 难点，以及我是怎么改的

这一节按“我本来以为是什么样 → 哪里不对 → 改了什么 → 现在是什么数”来写。选的是真卡住过我的地方。

4.1 测试全绿，功能没生效

这是我最想讲的一条，因为同一个形状在一天里出现了六次：代码看着对、测试全绿、功能根本没生效。

- 服务端校验上下文模式的白名单是 `camp / pve / pvp-live`，加本地对战时客户端开始发 `pvp-local`，白名单没跟着改。本地对战里的模型请求全部被 400 拒绝、静默回退成本地规则。规则建议照常工作，界面上没有任何异常——这个模式从头到尾没有产生过一次模型解释。
- 新增的 `rules.js` 没进服务端的静态资源白名单，`/rules.js` 返回 404，`app.js` 的 `import` 图整体失败，**整页白屏**。
- 陪练的 `notify()` 定义了，但全仓库没有一个调用点。主动气泡写完了，从不出现。
- 军师改造留下一个未定义变量，每秒抛一次 `ReferenceError`，浏览器日志里攒了 75 条。
- 对方面板把同一段内容重复显示了三处。
- 敌方补位“决定了却无人提交”：AI 选好了合法换宠、存进变量就返回，而玩家那条路径又拒绝替它提交。结果是界面永久冻结在“正在出招…”。

还有一个同类的：犹豫判定用的是墙钟时间，玩家中途去操作对方面板的 13 秒被算成了“犹豫了 13 秒”。

单元测试查不出这些，因为它们测的是函数本身，不测函数有没有被接上。所以补了两层检查。

第一层是 `wiring.test.js`，离线、确定性，查三件事：`app.js` 里每个 `$(‘id’)` 都必须在 `index.html` 或 `app.js` 自己建出来的元素里存在；`app.js` 的死函数清单必须恰好是两个已知遗留，多一个就失败；面试题要求的每项能力（军师局内提示、整局复盘、小测、聊天面板、陪练主动气泡）都必须在 `app.js` 里找得到入口。它当场就找出一处指向已删除元素的残留引用。

第二层是 `npm run test:smoke`，headless Chrome 真跑一局训练赛，断言营地渲染出 12 张伙伴卡（说明模块图活着）、比赛能打到结果、控制台零报错。变量未定义这

类错误只有真页面看得见。它故意不进 `npm test`——需要浏览器的检查不该当成所有人的必过门。

这一层也有它抓不到的东西，我自己就见过一次假阳性。冒烟测试第一次跑出“最长连续无可点 14 次”，我以为快卡死了。查下去发现是它从没设置播放速度，每一回合都在放动画、按钮本来就禁用——这个指标量的是动画时长，不是死锁。改成先设最快速度、并且只在横幅不显示“正在”的时候计数，同一个场景现在是 0。

4.2 玩家不打开聊天，就等于没有教练

第一版做出来我自己试了两局，发现一件事：小芽的所有能力都在聊天面板里，而我全程没打开过那个面板。一个要玩家主动召唤才存在的助手，在真实游玩里等于不存在。

改法是把主入口从聊天框挪到游戏事件上：开局和关键风险变化时，战斗页底部出现一行短提示，解释收在二级入口里按需展开；玩家出招之后，旧提示立刻隐藏，等新局面重新评估。聊天框降级成补充入口。

副作用是提示一多就变成噪音。所以门控最后比内容更难做：玩家是否在前台、是否已经出招、是否已经忽略过、这个回合是否已经提示过、上次提示距今多久、当前风险是否构成可行动的风险，逐项判断；每局总量、同类提示只出一次、冷却时间各有上限，老师和军师还共用同一份每局配额。

这里有一条我从教育数据挖掘的文献里学来的规则。那篇讲 *assistance dilemma* 的论文里，作者报告他们早期的预测器没有把“学生用了提示”这件事算进去，结果因为提示本身会把人推向高效步骤，用过的学生反而被判定成“不需要帮助”，他们的修正就是把两者分开。

小芽里是同一件事：判断玩家是否掌握某个知识点时，只用他没有被提示过的时候做出的选择，代码里就是一个过滤条件。我是先按直觉这么设计的，后来才发现这个坑在文献里有记录。

4.3 数字不能让模型算

我一开始让模型看局面然后给建议，试出来的回答里数字是错的。不是偶尔错，是稳定地错：编出不存在的技能名，算错伤害，在两个都能击倒的技能里挑面板数字大的那个。

后来把职责切开——引擎枚举、模型解释，也就是前面军师那一节讲的架构。这之后还剩两类错误是数字校验抓不到的。

第一类是道具名。游戏里只有回复药、净化药、能量果三样，模型会把净化药说成“解药”、能量果说成“以太”。数字全对，引用全对，只有名字是错的，因为没有数字可校验。

第二类是因果。宠物倒下时，它这一回合已经排好的行动会被取消，模型会把这次取消描述成“命中了”。同样没有数字可比。

这两类都靠加检查解决：维护一份别名黑名单，命中就判不合格；读回合事件，某个行动被记为取消时，正文里就不能出现这个行动造成了伤害。判不合格之后走既有的降级路径，显示规则结论而不是错误描述。

顺带说一处同类的文案问题。降级提示原来会把服务端原文（“教练上下文无效”）和内部错误码直接端到玩家面前。现在按原因映射成四句玩家话，内部代码只进日志。后台术语那件事我人工扫了三轮，三轮都漏：第一轮漏了若干处，第二轮又漏 5 处，第三轮才发现知识卡自己的 `principle` 正文里也有一个。靠一次扫除是扫不干净的。

所以这轮把它改成了机械检查。`evals/player-copy.test.js` 遍历玩家能读到的每一个字符串——军师短句、点开“看看原因”后的依据、被引用的知识卡正文、阵容建议、局内提示、收尾提示、整局复盘——命中禁止词表（“启发式”“枚举”“分支”“平均收益”这类）就失败。我做了负向验证：把“启发式”植入一句可达文案，测试失败；移除，测试通过。这一步是必须做的，因为**一个不会失败的检查比没有检查更糟**——它会让人以为这件事已经守住了。

这个检查刚写出来时是有缺口的，而且是我自己没看出来的：它用 `observe()` 取局内提示时没有传 `incident`，所以“明显策略错误”那条理由对应的正文不在覆盖范围内——那段正文里正好还留着一句“一回合启发式评分”。写报告的 `agent` 复现了这条路径并把它写进了报告，没有绕过去。

我补它的第一次尝试是错的。我想当然地手工造一个 `incident` 传给 `observe()`，负向验证立刻戳穿：把违规词放回源码，检查照样通过——因为那段正文根本不走 `observe()`，它由 `strategistTrigger` 在真的算出一次枚举事故时才生成。构造那条路径需要真实的局面和结算后快照，成本高，而且构造得出来也仍有别的分支够不着。

所以换成静态扫描：把 `coach/` 下所有模块里的中文字符串字面量逐个取出来对词表。它不依赖任何运行时路径，覆盖全部分支；代价是分不清“玩家可见”与“内部字段”，所以 `method` 与 `instruction` 这两个确实是内部用途的字段被具名豁免，而不是把规则放宽。

让它可信又花了三次修正，每一次都是跑出来才发现的：它把 `profile.tokens` 里的 `token` 当成违规词（变量名不是文案，现在先剔除 `${...}` 插值再判）；它把命中报在根本没有违规的行上（跨行配对反引号会错位，改成逐行取）；豁免最初没写住，两处内部字段一直报错。改成静态扫描之后，它又扫出四处运行时检查没覆盖到的真泄漏：老师复盘里的“平均分/最坏分”、卡片提示里的“对手分支”、以及两处“枚举”。

同一类问题还有一处不是术语、而是叫法：同一个动作，我方页签写“逃跑”、对方页签写“认输”，面板说明写“撤退”，结算横幅又写“已撤退”——一个动作四个名字，全是我自己写的。现在统一成“认输”。它和术语泄漏是同一个毛病：同一件事在两个地方说法不同，而这类不一致单元测试永远发现不了，因为每一处单独看都是对的。

4.4 “该不该调工具” 不该由模型决定

工具决策我做过三轮真实评测，同一个预注册用例集、44 条、全部真实 API 调用，每轮都保留原始数据，包括变差的那一轮。三轮依次改的是：原始提示词（“先检查已有证据，再决定是否补证据”）；把“默认是停止”写进提示词并点名哪几类必须调；以及给模型加一个“必须调用”的字段。第 3 轮已回退，第 2 轮是仓库当前的状态。

指标	第 1 轮	第 2 轮	第 3 轮
多余工具调用率	75.0%	33.3%	13.6%
漏调率	0.0%	21.9%	77.3%
严格正确率	31.25%	50.0%	31.8%
模型自停率	3.23%	96.8%	—
延迟 p50	4284 ms	2798 ms	2154 ms
44 条成本	\$0.0335	\$0.0209	\$0.0178



原文里“先检查再决定”这句话把“去查”当成了默认动作——12 条本该零调用的用例里 9 条调了工具，31 次决策里只有 1 次是模型自己判断停下来的。

第 3 轮为什么变差值得单独说。我加那个字段本意是补上三类硬需求，实际上等于告诉模型“这个字段是 null 就不必调”——而它在大多数用例上是 null。整个判断被推给了那个字段，漏调因此从 21.9% 跳到 77.3%。这是典型的过度规定：用一个新字段表达“必须调”，同时无意中表达了“其余都不必调”。

三轮的摆动说明问题不在措辞，在分工。数据给出的分界很清楚：模型**选哪个工具**很准（第 1 轮 90.63%；第 2 轮在“预期应调且确实调了”的 12 条上是 12/12），但**判断该不该调**只有一半正确。既然一半是错的，就不该让模型做这个判断。所以第 4 轮不是改提示词，是把政策写进代码（`coach/runtime.js` 里的 `policyFor`）：证据包里已经带着当前局面和最近回合事件，问当前状态的不用再查；只有结构上不含的四类才需要查。判定需要时，代码直接执行第一步，之后再问模型要不要继续查；判定不需要时，整个工具循环跳过。模型现在只在“还要不要继续看”这个方向上参与。

边界要说清。44 条里只有 32 条可判定，每格 1 条、单次运行、没有重复测量，所以上表所有差值都没有置信区间；第 2 轮和第 3 轮之间 18 个百分点的差距，有多少是真实效应、多少是单次波动，这批数据回答不了。而且严格正确率即使在第 2 轮也只有 50%。另外还有一处规格分歧我主动交代：评测的预注册标准认为“我现在场上这只还剩多少血”应该调一次 `read_state`，但证据包里本来就带着当前局面，调它只是把已有的事实再取一遍，第 2 轮那 7 条“漏调”里有 6 条属于这一类。所以“50%”里有相当一部分不是模型错，是两种政策的差异。产品必须选一个并说清楚，我选了“包里已有的事实不重复取”；按新口径重新标注必须在看到结果之前完成，这一轮我没做。

4.5 检索没有赢过关键词

我做过一个假设：把检索从关键词换成“关键词加多语言向量”的混合方案，口语化提问的命中率会提高。为了验证它，我建了一个四臂对照：不检索、纯关键词、混合，外加一个把反例文本提为一等字段的第四臂。128 条自查查询，112 条有正解、16 条越界，离线跑，不调用模型。

结果是没有任何一个臂显著赢过关键词基线。Hit@3 分别是 0.00%、88.39%、90.18%、91.07%，最好的第四臂相对关键词只高 2.68 个百分点，McNemar 精确检验 $p = 0.4531$ ，配对 bootstrap 95% 置信区间 $[-1.79, +7.14]$ 个百分点，跨零。按这个不一致对的比例，要有 80% 把握检出这个量级的差异，大约需要上千条查询。

这里我差点犯一个错。看到混合没赢，我的第一反应是“检索没用，RAG 是伪需求”。把四臂拆开看才发现不是：第四臂的 MRR 从 0.8007 升到 0.8244，排序确实好了。但这些改善都发生在 112 条正解上，一条查询的变动就能让 Hit@3 动将近一个百分点。所以我没有拿它证明任何事——既没有换成向量检索，也没有宣称 RAG 无效，而是记成“这次测量没测出差别”，把关键词保留为默认。

真正被这轮测出来的问题是负例拒答：三个检索臂都只有 56.25% (9/16)。“今天晚饭吃什么”“1+1 等于几”这类明显越界的提问仍然召回了卡片，原因是词项检索只要命中任意二元组就判为正分，没有最低分阈值，也没有拒答机制。这是未解决的问题，而且调权重解决不了。

还有一条边界必须写在前面：这 128 条查询是我自己写的，标注也是我自己读卡片文本后打的，不是独立盲标集，没有第二标注者，也没有一致性检验。一个直接证据是，沿用下来的那 20 条旧查询 Hit@3 是 81.25%，这次新写的 92 条是 93.48%——新写的明显更高，说明我出题时已经知道卡片里有哪些词。所以上面所有绝对数字都偏乐观，不能当外部基准。

4.6 模型会说过期的话

表现是：玩家已经出招了，模型那边的解释才回来，界面上出现一句针对已经过去的局面的建议。

我试过在提示词里写“如果局面已经变化就不要回答”，没用。模型看不到自己那句回答会在什么时候被显示出来，这个判断只能由程序做。

改法是把每个请求绑定到具体的局面版本：请求发出时记下对局 ID、回合号、规则版本和上下文 epoch，返回时逐项比对，任何一项变了就丢弃结果，不显示也不播报。客户端断连时服务端同时中止上游请求。这条有 6 项可复现的测试，用替换请求的方式构造慢响应，验证“快速出招后不显示旧文字、不重复补发”。真实浏览器里叠加慢响应这一半没做，需要可控的慢网环境，和语音的取消验证一起挂着。

4.7 对手原来不交给模型，后来交给了模型

题目场景是 PVP，我一开始想让模型来当对手，理由是“这样显得 AI 更强”。查了资料之后放弃了这个想法。

放弃的理由不是我懒，是证据方向相反。PokéLLMon 是一个纯靠语言模型打宝可梦的工作，阶梯赛胜率 49%。PokéChamp 换成“LLM 辅助 minimax 搜索”之后，对最强规则型 bot 胜率 84%，而且不需要额外训练。Metamon 进一步用离线强化学习加 Transformer，论文里说它的表现超过了 LLM 智能体和强启发式搜索引擎。在这个游戏类型里，决定胜负的是搜索和训练，不是语言能力。所以对手出招我用了引擎的启发式：轻松档随机、标准档优先伤害和治疗、挑战档枚举双方合法行动并比较一回合结果。

这一轮我把它改了：**对手的决策交给模型，引擎只负责列出有哪些合法行动**。上面那段调研的结论没有变，变的是它回答的问题——它说的是“谁更强”，而这轮要解决的是“对面得是个人”。一个按伤害公式调出来的启发式，出招永远挑不出毛病，也永远不像人；难度只是它的参数，不是它的性格。所以引擎留下来做两件事：用 `legalActions` 给出合法行动列表，以及在不该由模型决定的时候兜底。真正做选择的换成了模型，从那份列表里挑一个。

`chooseEnemy()` 因此降级成纯兜底，只在四条路径上出现：模型未配置（约 1 毫秒返回）、模型选了列表外的行动、超时（4 秒封顶）、网络错。这四条各自有测试，而且都必须能在没有后端、没有密钥的前提下工作。

难度改成只控制“给对手多少帮助”，不控制谁在决策。简单档只给合法行动列表；标准档多给一句对手自己教练的建议；挑战档再给出这一回合的枚举分数。同一个模型，三档拿到的是三份信息量不同的简报。这里有个复用值得一提：对手那个“自己的教练”不是另写一套，是拿同一份军师推理函数、把 `player` 和 `enemy` 对调后跑出来的，所以它读到的是自己的血量、能量和技能。对手和玩家用的是同一个大脑，只是坐在对面。

对手的教练在界面上仍然显示，但做了**高斯模糊**：看得见它存在，读不出内容，像隔着毛玻璃看别人的屏幕。真人同机分屏时不糊——那本来就是两个人共用一块屏。这里要说清一件事：模糊是观感手段，不是安全边界。对手教练的建议本来就基于双方公开的行动，不是隐藏信息，把它糊掉只是不想让玩家读着一句话打牌，不是防作弊。

实测（真实模型，服务在 8765，四次对照合计 543 次决策）：往返 p50 约 1.05 秒、p90 约 1.25 秒、最大 1.6 秒。玩家按正常节奏思考时**每回合等待 0 毫秒**——对手思考的那一秒被玩家自己的思考时间吸收了，界面上不出现“对手正在思考...”。玩家秒点时才会真等到，实测 0.89 到 2.52 秒。

兜底路径里只有“上游超时”真的触发过，而且只在并发压力下触发：另一批批跑时 135 次请求有 36 次撞上 4 秒上限，兜底率从 0 跳到 27%，延迟 p90 被抬到 3.7 秒。所以这条链路的真实上限不是“模型有多快”，而是“上游在并发下有多稳”，4 秒超时是它唯一的安全垫。模型未配置、选择非法、网络错这三条一次都没被真实流量碰到，目

前只有单测保证。

胜率这一项我没测出结论，原因比“样本小”更具体。挑战档下 agent 与引擎的选择 93% 相同（142 次里 10 次不同），标准档下 48% 不同（142 次里 74 次不同，两轮独立复跑都是这个量级）；但把 24 局放在一起看，**对手一局都没赢过**——四个格子两边的胜率是 100/100、100/100、66.7/50、50/50，这个阵容下胜负根本分不开两臂。所以准确的说法是：n=6 而且对手全败，这套对照既不能支持“对手变强了”，也不能支持“打平了”，它只是没有变弱的迹象。要拿到结论得换更强的对手阵容，或者改成看“偏离引擎的程度”这类指标，而不是看胜负。

有一条顺序保证是刻意做成结构而不是提示词约束的：每回合开始时先让对手独立做决定——它读的是回合前的公开局面——然后才开放我方的操作面板。它决定的时刻早于我的选择，所以不可能参考我的选择。对方面板会直接写明“已独立出招（看不到你的选择）”，按钮是禁用的，不是点了没反应。

也顺手记一个由此产生的设计问题。“电脑对手”和“训练模式”在玩法上其实是同一个东西——同一套 AI、同一套结算，区别只在教练闭不闭嘴。这个区别是真的，但我原来把它藏在名字后面，第一次给用户看的时候被直接问了“这两个有啥区别”。现在两个入口的名字直接写明差别：训练是“按关卡挑战”，对局是“对手自由配队”。

这一轮抓到两个真 bug，两个都不是单元测试发现的。

第一个：训练模式下 `decideEnemyFirst` 只在分屏那条分支被重新调用，于是从第 2 回合起复用了上一回合锁定的行动。表现是对手一直出同一手，对局打不完。单测全绿，因为被复用的那个行动本身完全合法。它和本节开头那六个是同一个形状——测试测的是函数对不对，不测函数有没有在正确的时机被调用。

第二个更该记：玩家补位的那一回合本来就不该出招，却被统计成了一次“引擎兜底”。代码没坏，是口径错了，而错的方向恰好让这套 agent 显得比实际更差。发现它之后我把补位回合从统计里摘掉，前面“兜底 0 次”才是干净的。测量本身也会说谎，这一条比第一个 bug 更值得记。

4.8 教练在 PVP 里该不该说话，我改错了两次

这一条是我这次最值得记的。

题目对军师的描述是“在对战 / 阵容场景给出决策建议”，而场景设定写的是 PVP 对战。也就是说军师本来就该在对战里工作。

我第一版的做法是：只要是对局模式，教练一律不介入。理由是“两人共用一块屏，给一方提示就是不公平”。这个理由听起来成立，我还把“公平性设计”写进文档当亮点。问题是这等于把主功能在题目指定的场景里禁用了。改法是把判据从“是不是对局模式”换成“有没有第三方受影响”：对手是真人时闭麦，对手是电脑时照常工作。

改完又有人指出这个版本也不自洽：电脑对手存在的意义就是模拟真人，那它和真人局的行为就该一致，否则这个模拟本身就是假的。这句话是对的。最终改成本地对战一律允许教练，只保留线上竞技的闭麦开关。真正需要守住的底线不是“整局不说

话”，而是“不读取对方待执行的动作”——那条项目一直没破，也有测试。

前后改了三次，前两次都是我把自己的推论当成了结论。

既然本地对战两边都能用教练，公平性就不需要靠禁用某个功能来实现了。做法是双方各有一条教练条，跑同一套引擎、同一套规则，各自只分析自己那一侧。对面用的是把 player 和 enemy 对调的镜像上下文，所以它看到的是对方的局面，也读不到任何隐藏信息——待执行的动作本来就不在上下文里。实测两条教练在同一回合给的结论不同，而且各自正确：一方建议“火花”，另一方建议“换上晶角犀”。

分屏这一块还踩过一个坑：两侧走不同的渲染函数，右边多一行标题，导致右侧的标签、按钮、教练条整体错位约 30 像素。改成两侧共用同一个渲染函数之后对齐了。

4.9 换一张表要沿着所有抄写它的地方走一遍

原来的属性相克表是不对称的：岩克三个属性，火和风各克一个，雷只被一个克。我数完之后发现说不出它为什么是这个形状——它不是设计出来的，是攒出来的。

换成两个独立的三环：火克草、草克水、水克火；岩克雷、雷克风、风克岩；环与环之间中性。现在每个属性恰好克一个、被一个克，这是一条可以直接断言的规则性质，不用靠眼睛看。同系伤害也从 $\times 0.75$ 改成 $\times 1$ ：原来它和“打向克制你的属性”共用了同一个分支，但这两件事不是一回事——游戏设定里没有任何东西支持“打自己人伤害更低”。

换完之后我以为最麻烦的是代码，结果代码是最省事的：军师、经验层、规则页都从同一张表读，跟着就走了。不跟着走的是那些把规则抄成自然语言的副本。手写的那张边界卡里写着旧规则，我改掉了；扫第二遍才发现同一类问题换了一层还在——六张由脚本生成的属性卡里仍然写着“同属性或被反克时 0.75”，而它们正是检索喂给模型的地面事实。也就是说教练会照着一张引擎已经不再实现的规则回答，而测试全绿，因为没有任何测试盯这句话。

根因是那句话在生成脚本里是硬编码的字符串，不在引擎的表里。这一轮把它改完了：`scripts/build-knowledge.js` 现在从 `RULES.typeAdvantage` 与 `RULES.typeResist` 取数，生成出来的六张卡直接写着“克制 1.5；打向克制你的属性 0.75；其余（含同系）1”。改引擎即自动同步，不再有第二份可以写错的副本。

值得留下的是这条教训的形状：改一个常量时真正要跟着改的，不是引用它的代码——代码会自动跟着走——是那些把规则抄成自然语言的副本，而它们没有类型系统盯着，也没有测试盯着。手工找这类副本我漏了三次，最后一次是知识卡自己的 `principle` 正文。这类问题不该靠人眼找，该靠生成器只从一处取数。

平衡矩阵也是这轮用新表重跑的，数字在下一节。

5 我做错的

上面那节多半是“发现问题然后改对”，这一节讲三件我判断错了、并且错了有一段时间才意识到的事。

5.1 语音：连续三次判断错方向

玩家反馈语音有两个问题：播报完有一声爆鸣，而且第一次开启是普通话、后面变成粤语。

第一次我判断是“系统里有英文语音在读中文”。错了。用 `say -v '?'` 把系统语音列出来才看到，Eddy、Flo、Grandma 这些名字确实各有真正的中文版本，示例文本就是“你好！我叫 Eddy”。

第二次我改语音选择逻辑，优先精确匹配 `zh-CN`。也错了，实测两个中文语音都还是粤语。

第三次我把爆鸣归因于 `speechSynthesis.cancel()` 之后没有等待，加了 300 毫秒的间隔。还是不行。到这一步我才意识到前两次都是在没有证据的情况下改代码——看到一个现象就猜一个原因，改完让使用者去试，试完再猜下一个。

真正定位问题是从使用者提供的一条信息开始的：“一开始是对的，取消一次再开启就又是粤语加爆鸣。”中间唯一发生的事就是 `cancel()`。也就是说这个操作会持久破坏后续播报的声音选择，只有刷新页面能恢复；而 `cancel()` 在代码里有九个调用点，包括每次出招，所以实际对战里每回合都会触发一次。这解释了“只有第一次是对的”。

改法是把 `cancel()` 全部删掉，代价是已经开口的那一句不能中途掐断。但到这一步，验证的成本已经明显高于这个功能的收益：音频我没有办法自验，无头浏览器没有声卡，每一次判断都要占用使用者的时间。所以我最后的选择是**整体停用语音**，代码留在开关后面，文档里如实写明停用原因，不再对外宣称具备这个能力。

这件事的教训不是“要小心”，是在没有独立验证手段的情况下，**不要让使用者反复替我做实验**。第一次失败之后就该把变量拆开，做成一个可以一次问清的东西，而不是连猜三次。

5.2 一段 CSS 从来没有被写进文件

营地页的配招卡片排版很乱，名称、数值和说明挤在同一行。我看着截图以为是设计问题，准备重做布局。

实际原因是那段 CSS 根本没写进样式表。我把追加 CSS 的命令用 `&&` 串在了 `node --check` 后面，而那一刻 `app.js` 正好因为我刚打错一个括号而报错，于是 `cat >> style.css` 被跳过了——后面那句 `echo` 照样打印成功，我就以为写进去了。后果是技能卡退回成 `inline-block`，所有文字连成一行。我按“设计问题”去改，改了也没用。

教训很短：**用命令的退出码判断上一步是否成功，而不是用它的输出**。后面我改成逐个确认每个选择器真的落盘了。

5.3 两次重启后端，两次把密钥弄丢

密钥按设计只保存在后端进程的内存里，不落盘。我为了加载服务端改动重启了两次，两次都让使用者重新录入。第一次还可以说是不知道，第二次是明知道的——我甚至提前写了提醒。真正的问题是我没把“重启会清掉密钥”当成一个需要设计的东

西，而是当成一个每次都要打招呼的麻烦。

后来补了：客户端遇到 403 会自动重建会话并重试一次，错误信息也不再一律显示“暂不可用”，而是把真实原因（会话失效、鉴权失败、超时）带出来。另外加了一个启动脚本 `scripts/start.sh`，密钥从 macOS 钥匙串读，不再每次重启都重新录入——保存密钥这件事交回给操作系统，应用本身仍然从不写盘。这些都是补丁，根因是这个项目从头到尾没有一个稳定的运行方式。

还有一件同类的、更小的：有一次我看到一份评测结果文件存在，就以为真实模型评测跑完了；打开才发现那份文件只有两行，是更早一次局部验证留下的。后来整理清单时我对每一条都去看实际数据，而不是看文件名。

6 数字

下面每个数都是跑出来的，附了产生它的命令或文件。有波动或没有置信区间的，我写明了。

6.1 规模

项目	值
宠物 / 关卡 / 难度	12 / 5 / 3
属性	7 个（两个独立三环，组间中性）
技能 / 道具 / 携带物	23 / 3 / 3
每只宠物的配招空间	学 6 招选 4 招，15 种
知识卡	90 张（49 张自写战术卡 + 41 条从引擎导出）
自动测试	249 项通过（ <code>npm test</code> ，18 个测试文件）
模型上下文 / 最大输出	1,000,000 / 384,000 tokens
工作预算	200,000 tokens

上下文这一行是纠正过的。项目原来按 32768 做预算，比模型的真实窗口小约 30 倍——那是本地分词器文件里写的长度，不是 API 的上限。我去查了 DeepSeek 的官方文档，`deepseek-flash` 是 1M 上下文 / 384K 最大输出，于是把它记成两个有名字的常量。工作预算仍然定在 200K，比上限低：把 1M 填满要花的是钱和延迟，这跟模型能接受多少是两个决定。

6.2 工具决策三轮真实评测

完整表格在“难点”一节，这里只记两条最该被追问的数。把“默认停止”写进提示词之后，多余工具调用率从 75.0% 降到 33.3%，模型自停率从 3.23% 升到 96.8%，严格正确率从 31.25% 升到 50.0%——但漏调从 0 升到 21.9%。这不是纯改善，是换了一种错法。再往后加“必须调用”字段的那一轮更差（漏调 77.3%），已经回退。44 条用例里只有 32 条可判定，每格 1 条、单次运行，所以这些差值都没有置信区间。

6.3 检索四臂对照

128 条查询，112 条有正解、16 条越界。离线跑，不调用模型。

臂	Hit@3	MRR	负例拒答
不检索	0.00%	0.0000	16/16（平凡结果）
纯关键词（现默认）	88.39%	0.8007	9/16
混合（词项 + 语义）	90.18%	0.7996	9/16
反例字段加权	91.07%	0.8244	9/16

最好的一臂相对关键词基线高 2.68 个百分点（112 条正解里 5 胜 2 负），McNemar 精确检验 $p = 0.4531$ ，配对 bootstrap 95% 置信区间 $[-1.79, +7.14]$ 个百分点，跨零。也就是说：没有证据表明它更好，只能说没有证据表明它更差。

两个必须一起读的口径。一是这 128 条查询全部由我手写、我自己标注，不是独立盲标集——沿用下来的 20 条旧查询 Hit@3 是 81.25%，新写的 92 条是 93.48%，高出的约 12 个百分点就是出题人知道答案的代价，所以绝对数字偏乐观。二是负例拒答三个检索臂完全一样，都是 56.25%，这是本轮真正测出来的问题，不是臂之间的排序。

6.4 平衡矩阵

用新的相克表重跑：5,796 场、398 个臂、181.6 秒，规则版本 0.6，脚本 `scripts/eval-balance-matrix.js`，不调用模型。胜负平是 1997 / 3764 / 35。

结论是**没有哪只宠物被证明明显更强**。同属性的六组配对在主要对照里全部未分离；旧规则下唯一统计显著的同类失衡（雷系两只之间相差 33.3 个百分点、置信区间不重叠）在新表下缩到 8.3 个百分点，区间重叠，消失了。九套阵容的胜率极差也从旧表的 63.0 个百分点收窄到 35.2 个百分点。

能确定的只有节奏效应，以及一堆“这批数据回答不了”：每格 12 到 60 场，小于约 12 到 15 个百分点的胜率差分不开；玩家侧是 8 个固定脚本策略，不是真人；全部对局 1 级、0 加点。所以矩阵能证明的是“那条显著差距消失了、离散度收窄了”，**不能证明“属性平衡整体变好”**——那需要跨属性对照和消融实验。这是我自己报告里的原话，我没有把它读成好消息。

6.5 两个训练实验

两个都不是“训练了 DeepSeek”，边界写在表里。

各有两处必须说明的地方。干预策略的回报更高，但它的**帮助次数反而更少**（1.70 对 5.89 次每局），增量帮助也更低（0.86 对 1.38）——它赢在打扰成本，不是赢在教得更好，这两件事不能混着讲；而且这些都是模拟玩家的口径，不等于真人学习效果。另外这一轮跑在规则版本 0.5 上，比现在的 0.6 早一个版本，我没有重跑。工具选择实验的 16 条测试全部是英文、二选一、共 24 条训练样本，它证明的是“参数确实更新了、并且学到了这个窄任务”，不是“模型会选工具”。两个实验都做了奖励投机审计：总

	干预策略实验	工具选择实验
学的是什么	教练何时开口（沉默 / 短提示）	该调哪个工具
方法	表格 Q-learning	REINFORCE + KL 正则
规模	3 个种子 × 6000 局	冻结骨干，更新 1152 个参数
数据	5 类模拟玩家	24 训练 / 8 验证 / 16 测试 (英文二选一)
结果	回报 1.116 (规则策略 -1.113)	测试集 0.5 → 0.9375, 三个种子一致

是提示、从不提示、只帮容易场景、只按风险提示这四种策略都单独跑过，没有出现靠刷指标拿高回报的情况。

6.6 对手 agent 的实测

对手 agent 的对照：脚本 `scripts/eval-opponent-agent.js`，产物 `reports/opponent-agent.json`。四个格子（玩家难度 × 对手难度），每格两边各 6 局，全部真实 API 调用。玩家两侧完全相同，都是同一个脚本策略，所以差异只可能来自对手那一侧的大脑。下表是第二轮独立复跑的数字（第一轮口径相同，结论一致）。挑战档两边用同一套枚举分数，所以 agent 很少偏离引擎；标准档只给一句教练建议，选择就分叉到一半左右（第一轮 58.7%，这一轮 52.1%）。延迟两档接近，都在 1 秒上下；543 次决策合并后 p50 约 1.05 秒、p90 约 1.25 秒。

对手难度	与引擎选择不同	往返 p50/p90/max	兜底
挑战档	10 / 142 (7.0%)	1059 / 1305 / 2983 ms	0 次
标准档	74 / 142 (52.1%)	1091 / 1342 / 1562 ms	0 次

必须一起读的三条边界。一是“玩家”是脚本策略不是真人，它对两臂的发挥完全一样——这保证了对照干净，同时也意味着结果不能外推到真人。二是 $n=6$ 、四个格子共 24 局，胜负是 100% / 100% / 66.7% / 50%，而**对手一局都没赢过**：这个阵容下胜负根本分不开两臂，所以这套对照不能支持“变强”，也不能支持“打平”，只能说没有变弱的迹象。三是并发那一轮出现过 36 次 4 秒超时兜底（135 次请求），所以上表“兜底 0 次”成立的前提是上游没有被打满，换个并发条件就会变差。

浏览器一侧的等待时间另有一份实测（headless Chrome 真打，用的是临时脚本 `tmp/e2e-opponent.mjs` 的 LIVE 模式，不属于长期产物）：玩家思考 1.5 秒时，16 个回合的等待全部是 0 毫秒，agent 决策 16 次、兜底 0 次、控制台零报错；玩家秒点时每回合真等 0.89 到 2.52 秒。这一条不支持“设计得好”，它只是说明对手那一秒确实被玩家的思考时间覆盖了。

6.7 接线自检与浏览器冒烟

`wiring.test.js` 离线跑，查 id 存在性、死函数清单、以及五个角色入口是否连着；`npm run test:smoke` 用 headless Chrome 真打一局训练赛，断言营地渲染出 12 张伙伴卡、比赛打到结果、控制台零报错。后者故意不进 `npm test`。这两条的由来和各自的盲区写在“难点”一节。

6.8 真实模型调用

上面三轮评测的 44 条用例全部是真实 API 调用。除此之外的行为采集里，已验证的包括 PVP 前置拒绝在 3 毫秒内本地返回、不产生上游调用；已发现的缺陷是把净化药说成“解药”、能量果说成“以太”，而数字校验没有拦住——这一条直接导致了后来加的名称校验与因果校验。

7 还没做完的，以及接下来做什么

7.1 明确决定不做的

下面四项不是“还差一点”，是本次决定不做，理由写在这里。

语音。已经整体停用（`VOICE_FEATURE=false`），控件隐藏，代码留在开关后面。原因是它连续三次判断错方向，而我没有办法独立验证音频——无头浏览器没有声卡，每一次判断都要占用使用者的时间。恢复语音之前这一块没有可观察对象，也无法验收。

联网 PVP。本地对战是同一台设备上分屏轮流操作，服务端仍然接受客户端传来的状态快照，不是权威对局服务。真实竞技要求服务端下发可见状态和模式，否则改一个字段就能绕过限制。这条差距不是工程量问题，是架构问题。

LLM 权重训练。本机是 arm64，没有可用的 CUDA 后端，也没有云上 GPU 预算。仓库里记录的 `cloud_budget` 是 `not supplied`。

训练期原始轨迹的完整归档。核对后确认核心要求已经满足：三个种子的独立测试结果一致，切分与轨迹摘要在。唯一缺的是训练期原始轨迹的完整落盘，属于归档事项。这个实验的说服力取决于它是什么（1152 个参数、英文二选一、冻结骨干），而不是证据的厚度，所以我决定不做。

7.2 受条件限制，做不了的

这些不是决定不做，是缺外部条件。

慢网那一条要补一句：使用者曾在公交车上用不稳定的连接真实游玩过，过期建议的场景是实际发生过的，不是假想；缺的只是能在浏览器里稳定复现它的环境。客户端一侧已经用替换请求的方式验证了 6 项。语音的听感验证也挂在这里——本机没有音频输出，我无法自验。

7.3 接下来的动作

按可核查的方式写，每项都带完成标志。

项目	缺什么
无提示迁移验证	需要真人被试在多个情境里重复观察；机制已实现，缺的是数据
人工试玩与学习效果观察	需要真人被试。现在没有真人学习效果的数据，模拟奖励不等于真人学习
分屏双人完整验收	需要两名真人打多局；分屏逻辑我用脚本模拟点击验证过
浏览器慢响应验收	需要可控的慢网环境。本机延迟只有 2-4 秒，不够
独立盲标的检索评测集	现在 128 条是我自写自标，不是独立盲标，绝对数字偏乐观
独立大样本模型质量评测	需要独立标注集与 API 预算；现有真实样本不构成质量结论

动作	完成标志	预计
按新政策重新标注评测集并复测	重新标注在看见结果之前完成	半天
给检索加最低分阈值与拒答再对照	负例拒答率显著高于 56.25%	半天
补独立标注者，重做检索盲测	不是自出题自评测	1 天
把文案静态扫描扩到 app.js	界面文案与 coach 模块用同一份词表	1 小时
让对手 agent 的胜率对比有真人样本	不是 n=6 的脚本玩家	待定
录一段完整演示	五条链路各有一段可播放记录	半天

最后一项需要说明：这台机器上没有可见桌面和音频，我生成的是逐步截图序列加文字说明（在 `output/demo/` 与 `docs/DEMO-WALKTHROUGH.md`），它不是录像。要让评审看到真实操作过程，需要人在正常桌面环境下录一次。

8 复现

```

npm start # 127.0.0.1:8765
npm test # 249
npm run test:smoke # headless Chrome
npm run build:knowledge #
npm run eval:retrieval #
npm run train:intervention #
node scripts/eval-opponent-agent.js # agent

```



来源清单在 `knowledge/sources.json` 与 `docs/DOMAIN-RESEARCH.md` 两份文件里，官方文档、公开转载与我的推断分开标注，不混在一起。